SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1417 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: Total Marks: 50

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I N.Pujitha Naga Sai Lakshmi (192572119) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

1. Trace Lexical and Syntax Analysis

Consider the C statement:

```
result = (a + b) * c - 10;
```

Lexical Analysis

Lexical analysis is the first major phase of a compiler. It reads the source program as a sequence of characters and groups those characters into meaningful units called lexemes. Each lexeme is then converted into a token that can be passed to the syntax analyzer. For the given statement, the lexical analyzer scans from left to right and ignores unnecessary white spaces.

result → Identifier (ID)

= → Assignment operator (ASSIGN)

(→ Left parenthesis (LPAREN)

a → Identifier (ID)

+ → Arithmetic operator (PLUS)

b → Identifier (ID)

) → Right parenthesis (RPAREN)

* → Arithmetic operator (MULTIPLY)

c → Identifier (ID)

- → Arithmetic operator (MINUS)

10 → Integer constant (NUM)

; → Statement terminator / separator (SEMICOLON)

Token Stream

ID ASSIGN LPAREN ID PLUS ID RPAREN MULTIPLY ID MINUS NUM SEMICOLON

The token stream provides a compact representation of the source statement. The lexical analyzer therefore converts the original character sequence into tokens that are easier for the parser to process.

Syntax Analysis

The syntax analyzer, or parser, receives the tokens generated by the lexical analyzer. Its main purpose is to determine whether the token sequence follows the grammar of the programming language. For the given statement, the parser identifies an assignment statement whose left-hand side is result and whose right-hand side is an arithmetic expression.

Assignment → ID = Expression ;

Expression → Expression - Term | Term

Term → Term * Factor | Factor

Factor → (Expression) | ID | NUM

Using these grammar rules, the expression inside the parentheses is recognized first. The parser constructs a hierarchical syntax structure in which a + b forms one subexpression. That complete subexpression is then multiplied by c, and finally 10 is subtracted.

Operators, Operands and Delimiters

Operators: =, +, *, -

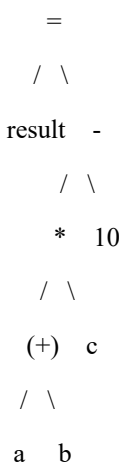
Operands: result, a, b, c and 10

Delimiters/separators: (,), ;

Here, result is the destination variable, while a, b, c and 10 participate as operands in the expression. The parentheses are delimiters that explicitly group the expression a + b, and the semicolon marks the end of the statement.

Operator Precedence and Associativity

Operator precedence determines the order in which different operators are considered during syntax analysis. Parentheses have the highest priority because they explicitly force the enclosed expression to be evaluated as a unit. Multiplication has higher precedence than subtraction. Therefore, (a + b) is formed first, then the result is multiplied by c, and only after that is 10 subtracted. Associativity determines the order when operators have the same precedence. The common arithmetic operators +, -, and * are left associative. The parser uses precedence and associativity rules to construct an unambiguous syntax tree.



Role of Semantic Analysis

After syntax analysis, semantic analysis checks whether the statement is meaningful according to the language rules. It verifies that result, a, b and c have been declared, that the identifiers have compatible data types, and that the arithmetic operations are valid. It also checks whether the final value can legally be assigned to result.

Thus, lexical analysis identifies the tokens, syntax analysis determines the grammatical structure, and semantic analysis checks the meaning and type consistency of the statement.

2. Analyze Lexical Ambiguity and Longest-Match Rule

Consider the statement:

```
value = a >= b && c != d;
```

Lexical Analysis

The lexical analyzer reads the statement character by character and identifies identifiers, assignment operators, relational operators, logical operators and separators. The resulting lexemes and tokens are:

value → Identifier (ID)

= → Assignment operator (ASSIGN)

a → Identifier (ID)

>= → Relational operator (GE)

b → Identifier (ID)

&& → Logical operator (AND)

c → Identifier (ID)

!= → Relational/equality operator (NE)

d → Identifier (ID)

; → Separator (SEMICOLON)

```
ID(value) ASSIGN ID(a) GE ID(b) AND ID(c) NE ID(d) SEMICOLON
```

Ambiguity between > and >=

There is a possible lexical ambiguity when the analyzer encounters the characters > and = together. The single character > is itself a valid relational operator, while the two-character sequence >= is another valid relational operator. If the analyzer selected > immediately without examining the next character, it would produce the wrong token sequence.

The longest-match rule solves this problem. The analyzer compares the possible patterns and selects the longest valid lexeme. Therefore, when it sees >=, it returns the single token GE instead of returning > followed by =.

Ambiguity between ! and !=

A similar situation occurs with ! and !=. If the language supports ! as a logical NOT operator, then ! is a valid one-character token. However, when ! is immediately followed by =, the two characters form the not-equal operator !=. The longest-match rule therefore causes != to be recognized as one token.

Suitable Token Specifications

IDENTIFIER : [A-Za-z][A-Za-z0-9]*

RELATIONAL_OP : <= | >= | == | != | < | >

LOGICAL_OP : && | || | !

ASSIGNMENT_OP : =

Order of Token Patterns

Overlapping patterns should be handled carefully. Longer operators should be considered before shorter operators. For example, >= should be recognized before >, <= before <, == before =, and != before !. This ordering avoids splitting a multi-character operator into two smaller tokens.

The same principle is commonly called maximal munch or longest match. It makes lexical recognition deterministic when several token patterns can begin with the same character.

Final Result

The complete token sequence is therefore:

ID ASSIGN ID GE ID AND ID NE ID SEMICOLON

This demonstrates how lexical rules and the longest-match principle allow the analyzer to correctly recognize operators even when their prefixes overlap.

3. Determine Token Stream and Handle Lexical Errors

Consider the program fragment:

```
int total = 250;
float average = total / 5.0;
result@ = average + 10;
```

Line 1 Analysis

The first line contains a declaration of the variable total and initializes it with the integer constant 250.

int → Keyword

total → Identifier

= → Assignment operator

250 → Integer constant

; → Separator

```
KEYWORD(int)  ID(total)  ASSIGN  INT_CONST(250)  SEMICOLON
```

The lexical analyzer recognizes int as a keyword because it is reserved by the language. total is an identifier because it is a user-defined name. The number 250 matches the integer constant pattern.

Line 2 Analysis

The second line declares average as a floating-point variable and initializes it using the division expression total / 5.0.

float → Keyword

average → Identifier

= → Assignment operator

total → Identifier

/ → Arithmetic operator

5.0 → Floating-point constant

; → Separator

```
KEYWORD(float) ID(average) ASSIGN ID(total) DIV FLOAT_CONST(5.0) SEMICOLON
```

The sequence 5.0 is recognized as a floating-point constant because it contains digits, a decimal point, and digits after the decimal point.

Line 3 Analysis and Lexical Error

The third line contains the sequence result@. The word result is a valid identifier, but the character @ does not belong to any of the token categories specified for this program.

result → Identifier

@ → Invalid character / lexical error

= → Assignment operator

average → Identifier

+ → Arithmetic operator

10 → Integer constant

; → Separator

ID(result) ERROR(@) ASSIGN ID(average) PLUS INT_CONST(10) SEMICOLON

Nature of the Error

The @ character is the lexical error. A lexical analyzer should identify the unexpected character, report it with its location, and use an error-recovery method so that scanning can continue. The analyzer should not silently treat @ as a valid identifier character unless the language specification explicitly permits it.

Error Recovery

A simple recovery strategy is to report the invalid character, discard it, and continue scanning from the next input character. In this case, after reporting ERROR(@), the analyzer continues with =, average, +, 10 and ;.

This recovery technique is useful because it allows the lexical analyzer to find additional errors in the same source program instead of terminating after the first invalid character.

Token Stream Before and After Error

Before error: ID(result)

Error: ERROR(@)

After error: ASSIGN ID(average) PLUS INT_CONST(10) SEMICOLON

Therefore, lexical analysis continues successfully after the error, while the invalid character is clearly reported to the programmer.

4. Design Regular Expressions for Token Recognition

The simplified language contains keywords, identifiers, integer constants, floating-point constants, assignment and relational operators, arithmetic operators and separators.

Regular Expressions

Keywords: (int|float|while|return)

Identifiers: [A-Za-z][A-Za-z0-9]*

Integer constants: [0-9]+

Floating-point constants: [0-9]+\.[0-9]+

Assignment operator: =

Relational operators: <=|>=|==|<|>

Arithmetic operators: \+|-|*|/

Separators: \(|\)|\{|\}|;

Input Program

```
float total = price * 12.5;
while (total >= 100) {
    total = total - 10;
return total;
```

Line-by-Line Tokenization

Line 1: float total = price * 12.5;

```
FLOAT_KW  ID(total)  ASSIGN  ID(price)  MUL  FLOAT_CONST(12.5)  SEMICOLON
```

The word float is first recognized as a word matching the identifier pattern, but the lexer checks it against the keyword table. Since float is reserved, it is returned as FLOAT_KW. The words total and price are not keywords, so they are returned as identifiers.

The number 12.5 is matched by the floating-point expression [0-9]+\.[0-9]+. Therefore it is returned as FLOAT_CONST and not as an integer constant.

Line 2: while (total >= 100) {

```
WHILE_KW  LPAREN  ID(total)  GE  INT_CONST(100)  RPAREN  LBRACE
```

The word while is a keyword. The parentheses and brace are separators. The operator >= is recognized as one relational token because the two-character pattern is matched before the single-character > pattern.

Line 3: total = total - 10;

ID(total) ASSIGN ID(total) MINUS INT_CONST(10) SEMICOLON

Both occurrences of total are identifiers. The equals sign is the assignment operator, the minus sign is an arithmetic operator, and 10 is an integer constant.

Line 4: return total;

RETURN_KW ID(total) SEMICOLON

Complete Token Stream

FLOAT_KW ID(total) ASSIGN ID(price) MUL FLOAT_CONST(12.5) SEMICOLON

WHILE_KW LPAREN ID(total) GE INT_CONST(100) RPAREN LBRACE

ID(total) ASSIGN ID(total) MINUS INT_CONST(10) SEMICOLON RBRACE

RETURN_KW ID(total) SEMICOLON

Keyword and Identifier Distinction

The regular expression for an identifier is broad enough to match words such as float, while and return. Therefore, lexical analysis normally recognizes the word first and then checks a reserved-word table. If the word appears in that table, it receives the keyword token. Otherwise, it receives the identifier token.

Floating-Point and Integer Distinction

An integer constant contains only digits and is matched by [0-9]+. A floating-point constant contains a decimal point between digit sequences and is matched by [0-9]+\.[0-9]+. Consequently, 100 is an integer constant while 12.5 is a floating-point constant.

This combination of regular expressions, keyword lookup and longest-match behavior enables the lexical analyzer to generate the complete token stream correctly

5. Analyze the Complete Compiler Processing

Consider the statement:

```
int average = total / count;
```

1. Lexical Analysis

Lexical analysis receives the source characters as input. It removes irrelevant spaces and identifies lexemes. Each lexeme is mapped to a token that represents its role in the source program.

int → Keyword (INT)

average → Identifier (ID)

= → Assignment operator (ASSIGN)

total → Identifier (ID)

/ → Arithmetic operator (DIV)

count → Identifier (ID)

; → Separator (SEMICOLON)

```
INT  ID(average)  ASSIGN  ID(total)  DIV  ID(count)  SEMICOLON
```

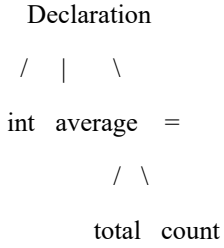
Output of lexical analysis is therefore a token stream rather than the original character sequence.

2. Syntax Analysis

Syntax analysis receives the token stream and checks it against the grammar of the programming language. It determines that the statement is a declaration followed by an initialization expression.

```
Declaration → Type ID = Expression ;
Expression → ID / ID
```

The parser constructs a syntax tree in which int specifies the declaration type, average is the declared identifier, and total / count forms the initializer expression. The division operator becomes an internal node with total and count as its operands.



3. Semantic Analysis

Semantic analysis checks the meaning of the syntactically correct statement. It verifies that average, total and count are properly declared and visible in the current scope. It also checks their data types and whether division is valid for those types.

Since average is declared as int, the compiler must ensure that the value produced by total / count can be assigned according to the language's type rules. If total and count are integers, the division may produce an integer result. If they are floating-point values, the assignment may require conversion or may involve a type compatibility rule.

Semantic analysis can also detect errors such as an undeclared identifier, incompatible assignment, or an invalid operation on non-numeric operands.

4. Intermediate Representation

After semantic analysis, the statement can be represented in a machine-independent intermediate form. A suitable three-address representation is:

```
t1 = total / count
average = t1
```

The temporary variable t1 stores the result of the division. Three-address code makes the operation simple for later optimization and target-code generation.

5. Code Generation

The code generator converts the intermediate representation into instructions suitable for the target machine. A generic register-based sequence could be:

```
LOAD R1, total
LOAD R2, count
DIV  R1, R2
STORE average, R1
```

These instructions are illustrative target-level instructions. The exact instruction names depend on the processor architecture. The important steps are loading the operands, performing division, and storing the result in average.

6. Symbol Table

The symbol table is a central compiler data structure that stores information about identifiers. During compilation, it can contain entries for average, total and count along with their data types, scope, storage location and other attributes.

During lexical analysis, identifier lexemes are recognized. During syntax analysis, identifiers are placed in their grammatical positions. During semantic analysis, the compiler looks up the identifiers in the symbol table to check declarations, scope and types. During code generation, storage information from the symbol table can be used to generate appropriate target instructions.

7. Compiler Organization

The complete compiler can be organized into a front end, middle end and back end.

Front end: Lexical analysis, syntax analysis and semantic analysis. It understands the source language and produces a validated intermediate representation.

Middle end: Performs machine-independent analysis and optimization on the intermediate representation. It attempts to improve the representation while preserving the meaning of the program.

Back end: Performs target-dependent code generation and produces machine or assembly instructions for the selected processor.

For the given statement, the front end transforms the characters into tokens, validates their grammatical and semantic correctness, and produces an intermediate representation. The middle end can analyze and optimize that representation. The back end then translates it into target-level instructions.

Overall Processing Summary

Source statement

↓

Lexical Analysis → tokens

↓

Syntax Analysis → syntax tree

↓

Semantic Analysis → type/declaration checked representation

↓

Intermediate Representation → t1 = total / count

↓

Optimization / Middle End

↓

Code Generation → target instructions

This phased organization makes compiler design systematic and modular. Each phase has a well-defined input and output, while the symbol table provides information required by several phases.